

LECTURE REPORT

CS50's Introduction to Cybersecurity

Lecture 3: Securing Software

Instructor	Institution	Module	Topic Focus
David J. Malan	Harvard University	Week 3 of 4	Software Security

1. Overview

This lecture focuses on securing software—both software that end users consume and software that developers build. The topics build upon previously introduced attack methods and extend them into the realm of web application vulnerabilities, injection attacks, client/server trust, and supply chain integrity.

2. Phishing

Phishing is a social engineering attack in which an adversary deceives a user into revealing sensitive information. A key mechanism exploited on the web is the HTML anchor tag, which separates the visible text from the actual destination URL.

Example of a deceptive anchor tag:

```
<a href="https://yale.edu">https://harvard.edu</a>
```

In this example, the link appears to point to Harvard University but actually navigates the user to Yale University's website. Adversaries use this technique to direct victims to counterfeit websites designed to harvest login credentials or other sensitive data.

Key insight:

- Always hover over a hyperlink before clicking to verify the actual destination URL displayed in the browser's status bar.
- Adversaries create convincing replicas of trusted sites to capture usernames, passwords, and financial information.

3. Code Injection & Cross-Site Scripting (XSS)

Cross-site scripting (XSS) is an attack in which malicious scripts are injected into a trusted website through user input. When a web application blindly reflects user-supplied data back in its output without sanitization, that data may be interpreted and executed as code by the browser.

Example malicious input:

```
<script>alert('attack')</script>
```

If an application echoes this input back into the HTML page without escaping, the browser executes the script. XSS attacks can be used to steal session tokens, redirect users, log keystrokes, or deliver further malware payloads.

4. Reflected vs. Stored XSS Attacks

4.1 Reflected Attack

A reflected XSS attack occurs when malicious input is sent to the server and immediately reflected back in the response. The attacker typically crafts a malicious URL and tricks the victim into clicking it. The victim's own browser then executes the injected script.

Example malicious URL (URL-encoded):

```
<a href="https://www.google.com/search?q=%3Cscript%3Ealert%28%27attack%27%29%3C%2Fscript%3E">cats</a>
```

The %3C and %3E are URL-encoded angle brackets. When decoded, the search query contains a script tag that the vulnerable application reflects and the browser executes.

4.2 Stored Attack

A stored XSS attack occurs when an application persists malicious code in its database and later renders it to other users. For example, a malicious script submitted as a comment on a blog post will execute in the browser of every user who subsequently views that comment. The impact is far broader than a reflected attack because it does not require individual targeting.

- Reflected XSS: One victim at a time, requires user to click a crafted link.
- Stored XSS: All viewers of the affected page become victims automatically.

5. Defenses: Character Escapes

Character escaping is the primary defense against XSS. Secure applications transform potentially dangerous characters in user input into their HTML entity equivalents before rendering them in the page. The browser then displays them as literal text rather than interpreting them as markup.

Unsafe output (vulnerable):

```
<p>About 6,420,000,000 <script>alert('attack')</script></p>
```

Safe output after escaping:

```
<p>About 6,420,000,000 &lt;script&gt;alert('attack')&lt;/script&gt;</p>
```

Commonly escaped characters:

HTML Entity	Character	Name
<code>&lt;</code>	<	Less-than sign
<code>&gt;</code>	>	Greater-than sign
<code>&amp;</code>	&	Ampersand
<code>&quot;</code>	"	Double quote
<code>&apos;</code>	'	Single quote / apostrophe

6. Defenses: HTTP Security Headers

HTTP headers sent by the server to the browser can enforce Content Security Policies (CSP) that restrict which resources the browser is permitted to load and execute. This provides a powerful additional layer of protection against XSS.

Restricting JavaScript to external .js files only:

```
Content-Security-Policy: script-src https://example.com/
```

Restricting CSS to external .css files only:

```
Content-Security-Policy: style-src https://example.com/
```

When a CSP is active, inline `<script>` or `<style>` tags embedded in the HTML will be blocked by the browser, even if an attacker successfully injected them. This significantly reduces the exploitability of XSS vulnerabilities.

7. SQL Injection

SQL injection (SQLi) is an attack that manipulates a database query by injecting malicious SQL code through user-supplied input. If user input is concatenated directly into a SQL statement without sanitization, the attacker can alter the query's logic.

Vulnerable query pattern:

```
SELECT * FROM users WHERE username = '{username}'
```

Attack 1 — Deleting all records:

Attacker input: `malan'; DELETE FROM users; --`

Resulting query:

```
SELECT * FROM users WHERE username = 'malan'; DELETE FROM users; --'
```

The semicolon ends the SELECT statement, and DELETE FROM users destroys the entire user table. The double dash (--) comments out the remaining SQL.

Attack 2 — Authentication bypass:

Attacker input in password field: ' OR '1'='1

Resulting query:

```
SELECT * FROM users WHERE (username = 'malan' AND password = '') OR '1'='1'
```

Since '1'='1' is always true, the WHERE clause always evaluates to true, returning every row in the users table. The attacker gains access to all accounts, including administrator accounts.

Core principle:

- Never trust user input. All input must be validated and sanitized before use in a query.

8. Defenses: Prepared Statements

Prepared statements (also called parameterized queries) are the primary defense against SQL injection. Rather than interpolating user input directly into SQL strings, the query structure is defined first with placeholders, and user input is bound separately.

Prepared statement form:

```
SELECT * FROM users WHERE username = ?
```

The database driver handles escaping automatically. For example, if the attacker inputs malan'; DELETE FROM users; --, the prepared statement renders it as:

```
SELECT * FROM users WHERE username = 'malan'; DELETE FROM users; --'
```

The single quote is escaped to two single quotes (""), meaning it is treated as a literal character within the string value rather than as SQL syntax. The malicious DELETE command is never executed.

- Prepared statements structurally separate code from data, making injection impossible at the query level.
- They should be used for every database query that incorporates external input.

9. Command Injection

A command injection attack occurs when user input is passed unsanitized to a system shell or command-line interpreter. If a web application calls functions such as system() or eval() with user-controlled data, an attacker can append arbitrary operating system commands.

Vulnerable functions to avoid with raw user input:

- `system()` — Executes a shell command directly.
- `eval()` — Evaluates a string as code in the running language.

Always consult the documentation for the language or framework in use to understand the correct escaping or sandboxing methods for any function that invokes system commands.

10. Developer Tools & the Danger of Client-Side Validation

Browser developer tools allow any user to inspect and modify the HTML, CSS, and JavaScript of a web page locally in their browser. Because HTML is rendered client-side, a user can alter attributes, remove restrictions, and submit modified data to the server.

Example — Removing a disabled attribute:

```
<input disabled type="checkbox">
```

After modification using developer tools:

```
<input type="checkbox">
```

Example — Removing a required attribute:

```
<input required type="text">
```

After modification:

```
<input type="text">
```

Client-side validation (HTML attributes like `required`, `disabled`, or `maxlength`) provides only a user experience convenience — it provides zero security. Any attacker can bypass it trivially.

11. Defenses: Server-Side Validation

Server-side validation is the authoritative security gate for all incoming data. Because the server controls its own execution environment, it cannot be modified by the client. Every piece of user input must be validated on the server before it is processed, stored, or used in any operation.

- Validate data types, lengths, formats, and ranges on the server for every request.
- Never assume client-side checks have run correctly or have not been bypassed.
- Treat every HTTP request as potentially adversarial, regardless of the apparent source.

12. Cross-Site Request Forgery (CSRF)

Cross-site request forgery (CSRF) is an attack that tricks a victim's browser into making an unintended, authenticated request to another website on the attacker's behalf. Because browsers

automatically attach session cookies to same-origin requests, the target server may honor the forged request as legitimate.

12.1 GET-Based CSRF

If a website uses GET requests for state-changing actions, an attacker can embed a malicious image tag that triggers the action automatically when the page loads:

```

```

No user interaction is required; simply loading the attacker's page fires the request.

12.2 POST-Based CSRF

POST requests can also be forged. An attacker hosts a page containing a hidden form that auto-submits on load:

```
<form action="https://www.amazon.com/" method="post">
<input name="dp" type="hidden" value="B07XLQ2FSK">
<button type="submit">Buy Now</button>
</form>
<script>document.forms[0].submit();</script>
```

12.3 CSRF Token Defense

The standard defense is a CSRF token — a cryptographically random, per-session secret value that the server embeds in each form or response. The server validates this token on every state-changing request. Because an attacker's cross-origin page cannot read the victim's token (blocked by the browser's same-origin policy), the forged request will fail validation.

- CSRF tokens are typically transmitted via hidden form fields or custom HTTP headers.
- Tokens must be unpredictable, unique per session (or per request), and verified server-side.

13. Arbitrary Code Execution (ACE)

Arbitrary code execution (ACE) refers to an attacker's ability to run code of their own choosing within a target process, bypassing the application's intended logic entirely.

13.1 Buffer Overflow

A buffer overflow occurs when a program writes more data to a fixed-size memory buffer than it can hold. The excess data spills into adjacent memory regions, potentially overwriting control structures such as return addresses. Attackers craft input of excessive length to redirect program execution to malicious shellcode.

13.2 Stack Overflow

A stack overflow is a specific form of overflow that corrupts the call stack. Attackers can use it to overwrite return addresses and inject executable code, gaining control of the process.

13.3 Cracking and Reverse Engineering

ACE techniques are also used for cracking software (bypassing license checks or payment walls) and for reverse engineering closed-source binaries to understand their internal logic or extract proprietary algorithms.

14. Software Supply Chain Security

14.1 Open-Source Software

Open-source software makes its source code publicly available. This enables community auditing — thousands of developers can review the code for vulnerabilities. However, it also means adversaries can study the code for weaknesses. Open-source security relies on the assumption that many eyes will identify and remediate issues faster than adversaries can exploit them.

14.2 Closed-Source Software

Closed-source software keeps its code proprietary and hidden from public view. This obscures implementation details from attackers, but it also means fewer external reviewers identify vulnerabilities. The security trade-off is security through obscurity versus the benefits of transparent community review.

14.3 App Stores

App stores operated by Apple (App Store) and Google (Google Play) act as trusted intermediaries. They require developers to sign their code with digital certificates, and the app store itself counter-signs approved applications. Operating systems then verify these signatures before installation, ensuring that only cryptographically authenticated, policy-compliant software can be installed through official channels.

14.4 Package Managers

Package managers (e.g., npm, pip, apt) use a similar signing mechanism for code libraries and system packages. Maintainers sign packages, and the package manager verifies signatures on download. While not foolproof — malicious packages have historically been published to repositories — signing raises the bar for supply chain attacks. Users should always prefer packages from well-maintained, audited sources.

15. Bug Bounty Programs

Bug bounty programs offer financial rewards to security researchers who discover and responsibly disclose vulnerabilities in software or systems. By providing a legal, structured channel for vulnerability reporting, bug bounties create an economic incentive for potential adversaries to report vulnerabilities rather than exploit or sell them.

- Programs are operated both publicly (open to any researcher) and privately (invitation-only).
- Payouts are typically scaled to the severity of the vulnerability discovered.
- They represent a cost-effective complement to internal security testing.

16. Identifying and Tracking Vulnerabilities

The cybersecurity industry maintains several standardized systems for cataloging, scoring, and prioritizing known vulnerabilities:

Acronym	Full Name	Purpose
CVE	Common Vulnerabilities and Exposures	A global registry of publicly known security vulnerabilities, each assigned a unique identifier.
CVSS	Common Vulnerability Scoring System	A standardized 0–10 numeric score that rates the severity of a vulnerability based on exploitability and impact.
EPSS	Exploit Prediction Scoring System	A probabilistic score estimating the likelihood that a given vulnerability will be exploited in the wild, helping defenders prioritize patching.
KEV	Known Exploited Vulnerabilities	CISA's authoritative list of vulnerabilities confirmed to have been actively exploited, used to mandate timely remediation in US federal agencies.

17. Summary

This lecture covered the major attack vectors targeting software and the corresponding defenses. The following table maps each threat to its mitigation:

Attack Vector	Mechanism	Primary Defense
Phishing	Spoofed hyperlinks / fake websites	User vigilance; verify URLs before clicking
XSS (Reflected/Stored)	Injected scripts via user input	Character escaping; Content-Security-Policy headers
SQL Injection	Malicious SQL via form fields	Prepared statements / parameterized queries
Command Injection	OS commands via unsanitized input	Input sanitization; avoid <code>system()/eval()</code> with user data
CSRF	Forged authenticated cross-origin requests	CSRF tokens validated server-side
ACE / Buffer Overflow	Memory corruption via oversized input	Memory-safe languages; bounds checking; ASLR
Malicious Software	Unsigned / untrusted code installation	App stores; package managers with signature verification

End of Lecture 3 Notes